D4 - First-Principles System Design

This is my final system design for the conversational memory project. It brings together the decisions and experiments I completed during D4.

1. What I am trying to build

The system should remember useful information from conversations without treating every past message as permanent memory. It also needs to handle changed preferences, old decisions, multiple users, sensitive information, and cases where no stored memory is actually useful.

The main idea is simple: memory is not just vector search. I need a memory record around the vector, and I need rules for what gets stored, what is still current, what belongs to which user, and what should be sent back into the model context.

2. Memory record

Each durable memory has a unique memory ID, user ID, memory text, embedding, memory type, provenance, creation time, and lifecycle status. When useful, I also keep subject, value, validity dates, and supersession links.

Part	Why I need it
memory_id	Keeps the metadata and FAISS vector tied to the same memory.
user_id	Makes ownership explicit and gives me a hard isolation boundary.
content + embedding	Content stores the meaning; the embedding is used for semantic retrieval.
type + provenance	Tells me what kind of memory it is and whether it was explicit or inferred.
lifecycle	Tracks active, superseded, or expired state.
supersession links	Keeps history without treating an old value as current.

3. What gets stored

I separated memory extraction from memory admission. Finding something that looks like a memory does not automatically mean I should store it.

Admission can return DURABLE_STORE, TEMPORARY_STORE, DON'T_STORE, or REJECT. The durable types I designed around are preference, fact, decision, and constraint. Temporary state normally stays temporary.

If something is uncertain, I do not keep it as durable memory. If it is sensitive under the admission rule, I reject it before durable storage.

4. Changed memories and conflicts

I did not want to overwrite old memories because that removes useful history. When the user clearly changes or corrects something, the new memory supersedes the specific older memory.

Current queries exclude superseded memories. Historical queries can still use them. If a value changes back later, I create a new memory instead of reactivating the old one.

For conflicts, I use a hierarchy instead of one weighted score: explicit supersession first, then provenance authority, then recency when the memories are otherwise comparable. Explicit user evidence has more authority than inferred evidence.

5. Retrieval and user isolation

The current design uses one shared FAISS index with IndexIDMap2 and explicit memory IDs. User filtering happens before candidate retrieval, not after it.

For each query, I first resolve the memory IDs owned by the requesting user. Then I use FAISS IDSelectorBatch so only those IDs can enter the candidate search. I tested this with another user's memory deliberately made more similar to the query, and it was still excluded.

6. Current vs historical retrieval

Current and historical questions should not use exactly the same eligibility rules. A current question normally uses active memories. A historical question may also use superseded memories.

I tested this using the FAISS to Qdrant example: the current query returned Qdrant, while the historical query could still recover the earlier FAISS memory.

7. No-memory behavior

The system should be allowed to return no memory. Nearest-neighbor search will always try to return something, even when the match is weak.

For D4 I tested a simple relevance threshold. If all candidates are below it, the result is empty. I still mark this part as PARTIAL because I have not tested the rule inside a complete end-to-end pipeline.

8. Context construction

Even good memories should not all be pushed into the prompt. I use a hard token budget and reserve space for the model response.

The prototype sorts candidate memories by relevance and greedily adds them while they fit. A memory can be skipped if it does not fit, and the final memory context can also be empty.

9. Memory lifecycle

The lifecycle has three states: active, superseded, and expired. A memory expires only when there is an explicit validity boundary such as valid_until. I am not using generic age-based decay.

Forgetting is separate from lifecycle state. A forgotten memory should stop being retrievable even if physical FAISS deletion fails. The deletion tests confirmed that logical exclusion can stay correct while cleanup is retried.

10. Privacy and forgetting

The main privacy rule is that one user must never receive another user's memory. The same ownership check is used for deletion, so a user cannot forget a memory they do not own.

Forgetting is idempotent. Deleting the same memory again should not create a new problem, and deleting a newer memory must not automatically reactivate an older superseded memory.

11. Evaluation

I reused the six failure cases from D3 instead of creating a new benchmark that was easier to pass. Cases 1 to 5 passed after the D4 design changes. Case 6, the unrelated cold-start case, is still PARTIAL because only the threshold behavior was tested in isolation.

D3 failure case	D4 result
Contradictory memories	PASS
Old + new decisions both retrieved	PASS
Too much historical context	PASS
Cross-user retrieval	PASS
Sensitive memory retrievable	PASS
Unrelated cold-start retrieval	PARTIAL

12. Engineering reality

I also tested whether the shared FAISS design is reasonable at larger sizes instead of assuming it would scale forever.

The 100K and 1M memory tests completed successfully. At 1M memories, latency increased as the authorized subset became larger. The 5M build failed on my development machine with std::bad_alloc, so I am not treating this setup as unlimited production-scale infrastructure.

For now, I am keeping the shared FAISS design because it is enough for the current project scope and I have evidence for it at 1M memories. I am not migrating just because a larger future architecture might eventually need something else.

13. Source of truth and recovery boundary

FAISS is only the vector-search index in this design. It is not the source of truth for memory ownership, lifecycle state, provenance, supersession, or forgetting. Those belong to the memory metadata layer.

I have not chosen or implemented the final persistent metadata store yet, so I cannot claim that the project already has a production source-of-truth database. For the current D4 scope, the design requirement is that the metadata layer is

authoritative and FAISS can be rebuilt from that durable state. The exact database and crash-recovery mechanism are left for later implementation work.

14. Consolidation and reflection

I am not adding automatic memory consolidation or reflection in the current design. The D4 failures I tested could be handled with admission rules, explicit supersession, lifecycle state, scoped retrieval, and context selection, so adding another background process would make the system more complex without evidence that I need it yet.

The consequence is that related memories are not automatically merged into summaries or rewritten into higher-level memories. If later testing shows that memory growth or repeated information becomes a real problem, consolidation can be added as a separate operation without changing the basic ownership and lifecycle rules.

15. What D4 does not claim

D4 gives me a system design and component-level evidence, not a finished production system.

I have not built production authentication, complete crash/restart recovery, unified observability, persistent transaction handling, or an integrated end-to-end latency benchmark. Those are real gaps, and I have left them as gaps instead of pretending they are solved.

16. Final design summary

The final design is a user-scoped conversational memory system where admission, lifecycle, retrieval, conflict resolution, context budgeting, and forgetting are separate responsibilities. FAISS handles semantic search, while the memory metadata layer is authoritative for ownership, lifecycle, provenance, supersession, and forgetting.

The main change from the naive baseline is that retrieval is no longer just 'find the closest vectors and pass them to the model'. The system now decides what deserves to be memory, who owns it, whether it is still current, whether the query is about the present or the past, and whether the memory is relevant enough to use at all.